Лабораторная работа № 3

по дисциплине
«Технологии промышленной разработки программного обеспечения.
Основные понятия»

Визуальный интерфейс к коллекции объектов на односвязном списке

Выполнил: студент группы М26-504

Гришанов Николай Вадимович

Москва — 2026

Содержание

1 Описание работы **3**

1.1 Цель и постановка 3

1.2 Что задано образцом и чего в нём нет 3

1.3 Принятое решение 3

2 Код **4**

2.1 Точка входа 4

2.2 Список, перечислитель и форма 4

2.3 Разметка окна 8

2.4 Проверка интерфейсов 10

3 Описание написанного кода **14**

3.1 MiniList<T> 14

3.2 Перечислитель 14

3.3 Form1 15

4 Описание тестовых сценариев **15**

5 Тестовые сценарии **16**

5.1 Старт 16

5.2 Добавление без списка 17

5.3 Создание списка 18

5.4 Перебор пустого списка 19

5.5 Ряд кнопок 20

5.6 Перебор кнопок с головы списка 21

5.7 Завершение прохода 23

5.8 Перебор объектов 24

5.9 Новый список продолжает нумерацию 25

5.10 Сценарии, закрытые только проверкой перечислителя 27

6 Документация **27**

6.1 Руководство оператора 27

6.2 Сборка 28

6.3 Ограничения 28

6.4 Состав поставки 29

Заключение **29**

1. Описание работы

1.1. Цель и постановка

Цель работы — сделать оконное приложение, в котором коллекция объектов живёт в оперативной памяти и видна на форме. Объекты заданного типа хранятся в односвязном списке. Оператор создаёт список, добавляет элементы и перебирает их. Перебор должен идти через стандартные интерфейсы IEnumerable<T>, IEnumerator<T>, IEnumerable и IEnumerator, а не через отдельный обход полей формы. По методическим указаниям тип элемента — CheckBox с видом кнопки. Кнопки стоят на панели с прокруткой. Смещение кнопки от левого края равно 40, умноженному на номер, который возвращает Append. В строке состояния показываются короткие сообщения о результате команды.

1.2. Что задано образцом и чего в нём нет

Образец задаёт класс MiniList<T> с вставкой узла в голову и четыре команды меню: «Создать список», «Добавить элемент», «Перебор кнопок», «Перебор объектов». Обработчики написаны только для первых двух команд. Класс ещё не реализует перечислитель, поэтому foreach по списку невозможен. Счётчик номеров объявлен как static int num. Он общий для всех списков одного закрытого типа и в образце не обнуляется. Поле AutoSize у CheckBox по умолчанию истинно, поэтому присваивание Width = 30 без отключения авторазмера не удерживает узкую кнопку из образца.

1.3. Принятое решение

MiniList<T> реализует IEnumerable<T>. Вложенный перечислитель вручную хранит текущий узел и выполняет MoveNext, Reset, Current и Dispose. Отдельного итератора через yield нет: в задании требуется проверить все методы обоих интерфейсов. Команда «Перебор кнопок» берёт перечислитель у необобщённого IEnumerable. Команда «Перебор объектов» берёт IEnumerator<CheckBox>. Оба проходят один и тот же список. Разница видна в строке состояния: «Кнопка: …» или «Объект: …». Текущая кнопка отмечается, панель прокручивается к ней. По завершении состояние отметок, которое было до команды, возвра-

щается, а в строке состояния остаётся «O.K.». Новый список заменяет старый: кнопки предыдущего списка снимаются с панели, потому что в новом списке их уже нет. Номер num при этом не сбрасывается, как в образце. Первая кнопка нового списка получает следующее смещение, а не ноль.

2. Код

Решение — два проекта SDK-стиля на .NET 10:

- Lab3 — окно Windows Forms и класс MiniList<T>;
- Lab3.Tests — консольная проверка перечислителя, пишет протокол.

Файл решения: Lab3.sln.

2.1. Точка входа

```
1  using System;
2  using System.Windows.Forms;
3
4  namespace Lab3
5  {
6      internal static class Program
7      {
8          [STAThread]
9          private static void Main()
10         {
11             ApplicationConfiguration.Initialize();
12             Application.Run(new Form1());
13         }
14     }
15 }
```

2.2. Список, перечислитель и форма

```
1  using System;
2  using System.Collections;
3  using System.Collections.Generic;
4  using System.Windows.Forms;
5
6  namespace Lab3
7  {
8      /// <summary>
9      /// Однонаправленный цепной список. Append вставляет узел в голову,
10     /// поэтому перебор идёт от последнего добавленного элемента к первому.
11     /// </summary>
12     public class MiniList<T> : IEnumerable<T>
13     {
14         static int num = 0;
```

```
15      public Node Top;
16
17      public class Node
18      {
19          public T content;
20          public Node Next = null;
21      }
22
23      public int Append(T s)
24      {
25          Node p = new Node();
26          p.content = s;
27          if (Top != null) p.Next = Top;
28          Top = p;
29          return num++;
30      }
31
32      public IEnumerator<T> GetEnumerator()
33      {
34          return new Enumerator(this);
35      }
36
37      IEnumerator IEnumerable.GetEnumerator()
38      {
39          return GetEnumerator();
40      }
41
42      private sealed class Enumerator : IEnumerator<T>
43      {
44          private readonly MiniList<T> list;
45          private Node current;
46          private bool started;
47
48          public Enumerator(MiniList<T> list)
49          {
50              this.list = list;
51          }
52
53          public T Current
54          {
55              get
56              {
57                  if (!started || current == null)
58                      throw new InvalidOperationException("Перечисление не начато или уже завершено.");
59                  return current.content;
60              }
61          }
62
63          object IEnumerator.Current
64          {
65              get { return Current; }
66          }
67
68          public bool MoveNext()
69          {
70              if (!started)
71              {
72                  current = list.Top;
73                  started = true;
74              }
75              else if (current != null)
```

```
            {
                current = current.Next;
            }

            return current != null;
        }

        public void Reset()
        {
            current = null;
            started = false;
        }

        public void Dispose()
        {
        }
    }
}

public partial class Form1 : Form
{
    MiniList<CheckBox> v;

    private Timer walkTimer;
    private IEnumerator walk;
    private bool genericWalk;
    private Dictionary<CheckBox, bool> savedChecked;

    public Form1()
    {
        InitializeComponent();
        walkTimer = new Timer();
        walkTimer.Interval = 300;
        walkTimer.Tick += walkTimer_Tick;
    }

    protected override void OnFormClosed(FormClosedEventArgs e)
    {
        остановитьПеребор(false);
        walkTimer.Dispose();
        base.OnFormClosed(e);
    }

    private void создатьСписокToolStripMenuItem_Click(object sender, EventArgs e)
    {
        остановитьПеребор(false);
        panel1.Controls.Clear();
        v = new MiniList<CheckBox>();
        toolStripStatusLabel1.Text = "Список создан.";
    }

    private void добавитьЭлементToolStripMenuItem_Click(object sender, EventArgs e)
    {
        if (v != null)
        {
            CheckBox vcb = new CheckBox();
            vcb.Appearance = Appearance.Button;
            vcb.FlatStyle = FlatStyle.Standard;
            vcb.AutoSize = false;
            vcb.TextAlign = System.Drawing.ContentAlignment.MiddleCenter;
            vcb.Top = 50;
```

```
137             vcb.Width = 30;
138             vcb.Height = 30;
139             vcb.Left = 40 * v.Append(vcb);
140             vcb.Text = Convert.ToString(vcb.Left);
141             this.panel1.Controls.Add(vcb);
142             toolStripStatusLabel1.Text = "";
143         }
144         else toolStripStatusLabel1.Text = "Список не создан.";
145     }
146
147     // Необобщённые IEnumerable и IEnumerator: подсветка идёт по звеньям списка.
148     private void переборКнопокToolStripMenuItem_Click(object sender, EventArgs e)
149     {
150         начатьПеребор(false);
151     }
152
153     // Обобщённые IEnumerable<CheckBox> и IEnumerator<CheckBox>.
154     private void переборОбъектовToolStripMenuItem_Click(object sender, EventArgs e)
155     {
156         начатьПеребор(true);
157     }
158
159     private void начатьПеребор(bool generic)
160     {
161         остановитьПеребор(true);
162
163         if (v == null)
164         {
165             toolStripStatusLabel1.Text = "Список не создан.";
166             return;
167         }
168
169         if (v.Top == null)
170         {
171             toolStripStatusLabel1.Text = "Список пуст.";
172             return;
173         }
174
175         savedChecked = new Dictionary<CheckBox, bool>();
176         foreach (CheckBox box in v)
177             savedChecked[box] = box.Checked;
178
179         genericWalk = generic;
180         walk = generic
181             ? (IEnumerator)v.GetEnumerator()
182             : ((IEnumerable)v).GetEnumerator();
183
184         walkTimer.Interval = savedChecked.Count > 24 ? 80 : 300;
185         if шагПеребора())
186             walkTimer.Start();
187     }
188
189     private void walkTimer_Tick(object sender, EventArgs e)
190     {
191         шагПеребора();
192     }
193
194     private bool шагПеребора()
195     {
196         if (walk == null || !walk.MoveNext())
197         {
```

```
            остановитьПеребор(true);
            toolStripStatusLabel1.Text = "O.K.";
            return false;
        }

        CheckBox box = genericWalk
            ? ((IEnumerator<CheckBox>)walk).Current
            : (CheckBox)walk.Current;

        foreach (KeyValuePair<CheckBox, bool> pair in savedChecked)
            pair.Key.Checked = pair.Key == box;

        toolStripStatusLabel1.Text = genericWalk
            ? "Объект: " + box.Text
            : "Кнопка: " + box.Text;
        panel1.ScrollControlIntoView(box);
        return true;
    }

    private void остановитьПеребор(bool restore)
    {
        walkTimer.Stop();

        if (walk is IDisposable disposable)
        {
            disposable.Dispose();
            walk = null;
        }

        if (restore && savedChecked != null)
        {
            foreach (KeyValuePair<CheckBox, bool> pair in savedChecked)
            {
                if (!pair.Key.IsDisposed)
                    pair.Key.Checked = pair.Value;
            }
        }

        savedChecked = null;
    }
  }
}
```

2.3. Разметка окна

```
namespace Lab3
{
    partial class Form1
    {
        private System.ComponentModel.IContainer components = null;

        protected override void Dispose(bool disposing)
        {
            if (disposing && (components != null))
            {
                components.Dispose();
            }
```

```
13          base.Dispose(disposing);
14      }
15
16      private void InitializeComponent()
17      {
18          this.menuStrip1 = new System.Windows.Forms.MenuStrip();
19          thisсоздатьСписок.ToolStripMenuItem = new System.Windows.Forms.ToolStripMenuItem();
20          thisдобавитьЭлемент.ToolStripMenuItem = new System.Windows.Forms.ToolStripMenuItem();
21          thisпереборКнопок.ToolStripMenuItem = new System.Windows.Forms.ToolStripMenuItem();
22          thisпереборОбъектов.ToolStripMenuItem = new System.Windows.Forms.ToolStripMenuItem();
23          this.panel1 = new System.Windows.Forms.Panel();
24          this.statusStrip1 = new System.Windows.Forms.StatusStrip();
25          this.toolStripStatusLabel1 = new System.Windows.Forms.ToolStripStatusLabel();
26          this.menuStrip1.SuspendLayout();
27          this.statusStrip1.SuspendLayout();
28          this.SuspendLayout();
29          //
30          // menuStrip1
31          //
32          this.menuStrip1.Items.AddRange(new System.Windows.Forms.ToolStripItem[] {
33              thisсоздатьСписок.ToolStripMenuItem,
34              thisдобавитьЭлемент.ToolStripMenuItem,
35              thisпереборКнопок.ToolStripMenuItem,
36              thisпереборОбъектов.ToolStripMenuItem});
37          this.menuStrip1.Location = new System.Drawing.Point(0, 0);
38          this.menuStrip1.Name = "menuStrip1";
39          this.menuStrip1.Size = new System.Drawing.Size(784, 24);
40          this.menuStrip1.TabIndex = 0;
41          this.menuStrip1.Text = "menuStrip1";
42          //
43          // создатьСписокToolStripMenuItem
44          //
45          thisсоздатьСписок.ToolStripMenuItem.Name = "создатьСписокToolStripMenuItem";
46          thisсоздатьСписок.ToolStripMenuItem.Size = new System.Drawing.Size(109, 20);
47          thisсоздатьСписок.ToolStripMenuItem.Text = "Создать список";
48          thisсоздатьСписок.ToolStripMenuItem.Click += new System.EventHandler(thisсоздатьСписок.ToolStripMenuItem_Click);
49          //
50          // добавитьЭлементToolStripMenuItem
51          //
52          thisдобавитьЭлемент.ToolStripMenuItem.Name = "добавитьЭлементToolStripMenuItem";
53          thisдобавитьЭлемент.ToolStripMenuItem.Size = new System.Drawing.Size(124, 20);
54          thisдобавитьЭлемент.ToolStripMenuItem.Text = "Добавить элемент";
55          thisдобавитьЭлемент.ToolStripMenuItem.Click += new System.EventHandler(thisдобавитьЭлемент.ToolStripMenuItem_Click);
56          //
57          // переборКнопокToolStripMenuItem
58          //
59          thisпереборКнопок.ToolStripMenuItem.Name = "переборКнопокToolStripMenuItem";
60          thisпереборКнопок.ToolStripMenuItem.Size = new System.Drawing.Size(112, 20);
61          thisпереборКнопок.ToolStripMenuItem.Text = "Перебор кнопок";
62          thisпереборКнопок.ToolStripMenuItem.Click += new System.EventHandler(thisпереборКнопок.ToolStripMenuItem_Click);
63          //
64          // переборОбъектовToolStripMenuItem
65          //
66          thisпереборОбъектов.ToolStripMenuItem.Name = "переборОбъектовToolStripMenuItem";
67          thisпереборОбъектов.ToolStripMenuItem.Size = new System.Drawing.Size(129, 20);
68          thisпереборОбъектов.ToolStripMenuItem.Text = "Перебор объектов";
69          thisпереборОбъектов.ToolStripMenuItem.Click += new System.EventHandler(thisпереборОбъектов.ToolStripMenuItem_Click);
70          //
71          // panel1
72          //
73          this.panel1.AutoScroll = true;
```

```
74          this.panel1.Dock = System.Windows.Forms.DockStyle.Fill;
75          this.panel1.Location = new System.Drawing.Point(0, 24);
76          this.panel1.Name = "panel1";
77          this.panel1.Size = new System.Drawing.Size(784, 415);
78          this.panel1.TabIndex = 1;
79          //
80          // statusStrip1
81          //
82          this.statusStrip1.Items.AddRange(new System.Windows.Forms.ToolStripItem[] {
83              this.toolStripStatusLabel1});
84          this.statusStrip1.Location = new System.Drawing.Point(0, 439);
85          this.statusStrip1.Name = "statusStrip1";
86          this.statusStrip1.Size = new System.Drawing.Size(784, 22);
87          this.statusStrip1.TabIndex = 2;
88          this.statusStrip1.Text = "statusStrip1";
89          //
90          // toolStripStatusLabel1
91          //
92          this.toolStripStatusLabel1.Name = "toolStripStatusLabel1";
93          this.toolStripStatusLabel1.Size = new System.Drawing.Size(0, 17);
94          this.toolStripStatusLabel1.Text = "";
95          //
96          // Form1
97          //
98          this.AutoScaleDimensions = new System.Drawing.SizeF(7F, 15F);
99          this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;
100         this.ClientSize = new System.Drawing.Size(784, 461);
101         this.Controls.Add(this.panel1);
102         this.Controls.Add(this.statusStrip1);
103         this.Controls.Add(this.menuStrip1);
104         this.MainMenuStrip = this.menuStrip1;
105         this.MinimumSize = new System.Drawing.Size(480, 280);
106         this.Name = "Form1";
107         this.StartPosition = System.Windows.Forms.FormStartPosition.CenterScreen;
108         this.Text = "Form1";
109         this.menuStrip1.ResumeLayout(false);
110         this.menuStrip1.PerformLayout();
111         this.statusStrip1.ResumeLayout(false);
112         this.statusStrip1.PerformLayout();
113         this.ResumeLayout(false);
114         this.PerformLayout();
115     }
116
117     private System.Windows.Forms.MenuStrip menuStrip1;
118     private System.Windows.Forms.ToolStripMenuItem создатьСписокToolStripMenuItem;
119     private System.Windows.Forms.ToolStripMenuItem добавитьЭлементToolStripMenuItem;
120     private System.Windows.Forms.ToolStripMenuItem переборКнопокToolStripMenuItem;
121     private System.Windows.Forms.ToolStripMenuItem переборОбъектовToolStripMenuItem;
122     private System.Windows.Forms.Panel panel1;
123     private System.Windows.Forms.StatusStrip statusStrip1;
124     private System.Windows.Forms.ToolStripStatusLabel toolStripStatusLabel1;
125 }
126 }
```

2.4. Проверка интерфейсов

```
1 using System;
```

```
using System.Collections;
using System.Collections.Generic;
using System.IO;
using System.Text;
using Lab3;

namespace Lab3.Tests
{
    internal static class Program
    {
        private static int passed;
        private static int failed;
        private static readonly StringBuilder report = new StringBuilder();

        private static int Main()
        {
            Console.OutputEncoding = Encoding.UTF8;

            report.AppendLine("Протокол тестирования MiniList<T>");
            report.AppendLine("Лабораторная работа №3. Интерфейсы IEnumerable<T>, IEnumerator<T>, IEnumerable, IEnumerator.");
            report.AppendLine();
            report.AppendLine("План");
            report.AppendLine("1. Пустой список: MoveNext возвращает false, чтение Current до начала запрещено.");
            report.AppendLine("2. Append вставляет в голову: порядок перебора обратный порядку добавления, номера узлов возрастают.");
            report.AppendLine("3. Обобщённый foreach и необобщённый foreach дают одну и ту же последовательность.");
            report.AppendLine("4. Reset возвращает перечислитель в исходное положение, повторный проход совпадает с первым.");
            report.AppendLine("5. Current после завершения перебора запрещён; лишние MoveNext остаются false.");
            report.AppendLine("6. Dispose не выбрасывает исключение. Два перечислителя одного списка независимы.");
            report.AppendLine();
            report.AppendLine("Результаты");

            TestEmpty();
            TestAppendOrder();
            TestGenericAndNonGeneric();
            TestReset();
            TestAfterEnd();
            TestDisposeAndIndependentEnumerators();

            report.AppendLine();
            report.AppendLine("Итог: пройдено " + passed + ", не пройдено " + failed + ".");
            report.AppendLine();
            if (failed == 0)
            {
                report.AppendLine("Заключение");
                report.AppendLine("MiniList<T> строит односвязный список с вставкой в голову и отдаёт элементы через обобщённый и необобщённый перечислители. Контракт Current, MoveNext, Reset и Dispose выполняется на пустом списке, на списке из нескольких узлов и при повторном проходе. Реализация интерфейсов пригодна для перебора коллекции в приложении.");
            }
            else
            {
                report.AppendLine("Заключение");
                report.AppendLine("Есть непройденные проверки. Реализацию перечислителя нужно исправить до сдачи.");
            }

            string text = report.ToString();
            Console.Write(text);
            File.WriteAllText(Path.Combine(Directory.GetCurrentDirectory(), "Протоколтестирования_.txt"), text, new UTF8Encoding(true));
            return failed == 0 ? 0 : 1;
        }

        private static void Check(string name, bool condition)
```

```
61      {
62          if (condition)
63          {
64              passed++;
65              report.AppendLine("  [+] " + name);
66          }
67          else
68          {
69              failed++;
70              report.AppendLine("  [−] " + name);
71          }
72      }
73
74      private static void CheckThrows(string name, Action action)
75      {
76          bool thrown = false;
77          try
78          {
79              action();
80          }
81          catch (InvalidOperationException)
82          {
83              thrown = true;
84          }
85          catch (Exception)
86          {
87              thrown = false;
88          }
89
90          Check(name, thrown);
91      }
92
93      private static void TestEmpty()
94      {
95          MiniList<string> list = new MiniList<string>();
96          using (IEnumerator<string> enumerator = list.GetEnumerator())
97          {
98              CheckThrows("Пустой список: Current до MoveNext", delegate { string unused = enumerator.Current; });
99              Check("Пустой список: MoveNext == false", enumerator.MoveNext() == false);
100             CheckThrows("Пустой список: Current после MoveNext", delegate { string unused = enumerator.Current; });
101         }
102     }
103
104     private static void TestAppendOrder()
105     {
106         MiniList<string> list = new MiniList<string>();
107         int first = list.Append("a");
108         int second = list.Append("b");
109         int third = list.Append("c");
110
111         Check("Append возвращает возрастающие номера", second == first + 1 && third == second + 1);
112         Check("Голова списка — последний добавленный элемент", list.Top != null && list.Top.content == "c");
113         Check("Второй узел хранит предыдущий элемент", list.Top.Next != null && list.Top.Next.content == "b");
114         Check("Хвост хранит первый элемент и не имеет продолжения",
115             list.Top.Next.Next != null && list.Top.Next.Next.content == "a" && list.Top.Next.Next.Next == null);
116     }
117
118     private static void TestGenericAndNonGeneric()
119     {
120         MiniList<int> list = new MiniList<int>();
121         list.Append(1);
```

```
122         list.Append(2);
123         list.Append(3);
124
125         List<int> generic = new List<int>();
126         foreach (int value in list)
127             generic.Add(value);
128
129         List<int> nonGeneric = new List<int>();
130         foreach (object value in (IEnumerable)list)
131             nonGeneric.Add((int)value);
132
133         Check("Обобщённый foreach: 3, 2, 1", generic.Count == 3 && generic[0] == 3 && generic[1] == 2 && generic[2] == 1);
134         Check("Необобщённый foreach совпадает с обобщённым",
135             nonGeneric.Count == generic.Count && nonGeneric[0] == generic[0] && nonGeneric[1] == generic[1] && nonGeneric[2] ==
        generic[2]);
136
137         IEnumerator raw = ((IEnumerable)list).GetEnumerator();
138         raw.MoveNext();
139         Check("IEnumerator.Current возвращает тот же объект, что и обобщённый Current", raw.Current is int && (int)raw.Current == 3);
140         (raw as IDisposable).Dispose();
141     }
142
143     private static void TestReset()
144     {
145         MiniList<string> list = new MiniList<string>();
146         list.Append("x");
147         list.Append("y");
148
149         using (IEnumerator<string> enumerator = list.GetEnumerator())
150         {
151             Check("Первый MoveNext", enumerator.MoveNext() && enumerator.Current == "y");
152             Check("Второй MoveNext", enumerator.MoveNext() && enumerator.Current == "x");
153             enumerator.Reset();
154             CheckThrows("После Reset чтение Current запрещено", delegate { string unused = enumerator.Current; });
155             Check("После Reset проход повторяется", enumerator.MoveNext() && enumerator.Current == "y");
156         }
157     }
158
159     private static void TestAfterEnd()
160     {
161         MiniList<string> list = new MiniList<string>();
162         list.Append("only");
163
164         using (IEnumerator<string> enumerator = list.GetEnumerator())
165         {
166             enumerator.MoveNext();
167             Check("Повторный MoveNext за последним элементом == false", enumerator.MoveNext() == false);
168             Check("Ещё один MoveNext остаётся false", enumerator.MoveNext() == false);
169             CheckThrows("Current после завершения запрещён", delegate { string unused = enumerator.Current; });
170         }
171     }
172
173     private static void TestDisposeAndIndependentEnumerators()
174     {
175         MiniList<string> list = new MiniList<string>();
176         list.Append("1");
177         list.Append("2");
178
179         IEnumerator<string> first = list.GetEnumerator();
180         IEnumerator<string> second = list.GetEnumerator();
181         first.MoveNext();
```

```
182             second.MoveNext();
183             second.MoveNext();
184
185             Check("Два перечислителя независимы", first.Current == "2" && second.Current == "1");
186
187             bool disposeOk = true;
188             try
189             {
190                 first.Dispose();
191                 second.Dispose();
192             }
193             catch (Exception)
194             {
195                 disposeOk = false;
196             }
197
198             Check("Dispose не выбрасывает исключение", disposeOk);
199         }
200     }
201 }
```

3. Описание написанного кода

3.1. MiniList<T>

Класс повторяет образец. Узел Node хранит значение content и ссылку Next. Поле Top — голова списка. Append создаёт узел, ставит его перед прежней головой и возвращает num++. Из-за вставки в голову порядок перебора обратен порядку добавления: последняя кнопка на панели оказывается первой в списке. GetEnumerator каждый раз создаёт новый перечислитель. Необобщённый метод интерфейса IEnumerable вызывает тот же обобщённый метод, поэтому оба меню обходят одни и те же узлы.

3.2. Перечислитель

Перечислитель помнит список, текущий узел и флаг started. До первого MoveNext флаг ложен, и чтение Current выбрасывает InvalidOperationException. Первый MoveNext берёт Top. Следующие переходят по Next. Когда узел кончился, MoveNext возвращает false, а Current снова запрещён. Повторный вызов за концом списка остаётся ложным. Reset возвращает перечислитель в положение «до первого элемента». Dispose ничего не освобождает: узлы принадлежат списку, а не обходу. Два перечислителя одного списка не мешают друг другу. Необобщённое свойство Current возвращает тот же объект, что и обобщённое.

3.3. Form1

На форме — строка меню из четырёх команд, панель panel1 с AutoScroll = true и строка состояния с пустой меткой. Это имена и свойства из указаний. «Создать список» останавливает идущий перебор, очищает панель и записывает в поле v новый MiniList<CheckBox>. В строке состояния — «Список создан.» Очистка панели нужна потому, что старые кнопки к новому списку уже не относятся. Счётчик num команда не трогает. «Добавить элемент» при пустой ссылке v пишет «Список не создан.» Иначе создаёт CheckBox с Appearance.Button и FlatStyle.Standard, отключает AutoSize, задаёт квадрат 30 на 30 и выравнивает подпись по центру. Левый край равен 40 * Append. Подпись — это значение Left, как в образце. Узкая кнопка не вмещает три цифры целиком, поэтому на кнопке видна только часть числа. Полное значение показывает строка состояния во время перебора. «Перебор кнопок» и «Перебор объектов» не стартуют, если списка нет или в нём нет узлов: сообщения «Список не создан.» и «Список пуст.» Перед проходом запоминаются отметки кнопок. Таймер с периодом 300 мс (80 мс, если кнопок больше 24) вызывает один шаг: MoveNext, отметка только у текущей кнопки, текст «Кнопка:» или «Объект:», прокрутка панели к этой кнопке. На последнем неудачном MoveNext перечислитель освобождается, отметки возвращаются, в строке состояния — «O.K.»

4. Описание тестовых сценариев

Проверяются два слоя. Перечислитель проверяется без окна. Программа Lab3.Tests создаёт MiniList<T> и вызывает те же методы, что форма вызывает из меню: Append, GetEnumerator, MoveNext, Current, Reset, Dispose, а также foreach по обобщённому и необобщённому интерфейсу. Команда из каталога лабораторной:

```
dotnet run —project Lab3.Tests
```

Результат прогона: не пройдено 0, пройдено 19. Текст плана, результатов и заключения записан в Протокол_тестирования.txt. Окно проверяется отдельным прогоном по командам меню. После каждого контрольного состояния сохраняется снимок всего окна. Снимки лежат в report/images и приведены ниже. Снимок фиксирует меню, панель, отметку текущей кнопки, положение прокрутки

и строку состояния. Повторная съёмка:

```
dotnet run —project report/_capture — report/images
```

Группы сценариев:

1. отказ добавления и перебора, пока список не создан;
2. пустой список: команда перебора не падает и сообщает, что список пуст;
3. ряд кнопок: смещение 40 * номер, горизонтальная прокрутка, ручные отметки;
4. перебор кнопок с головы списка, то есть с последней добавленной кнопки, затем шаг к более ранним;
5. завершение прохода: «O.K.» и возврат прежних отметок;
6. перебор объектов тем же порядком, но с подписью «Объект:»;
7. новый список снимает старые кнопки и продолжает нумерацию;
8. контракт перечислителя на пустом списке, на списке из нескольких узлов, после Reset и при двух одновременных обходах.

5. Тестовые сценарии

Ниже — сценарии, выполненные в окне. Ожидаемый текст строки состояния совпал с фактическим.

5.1. Старт

Шаги: запустить приложение.
Ожидание: меню из четырёх команд, панель пустая, строка состояния пустая.
Факт: совпало, рисунок 1.

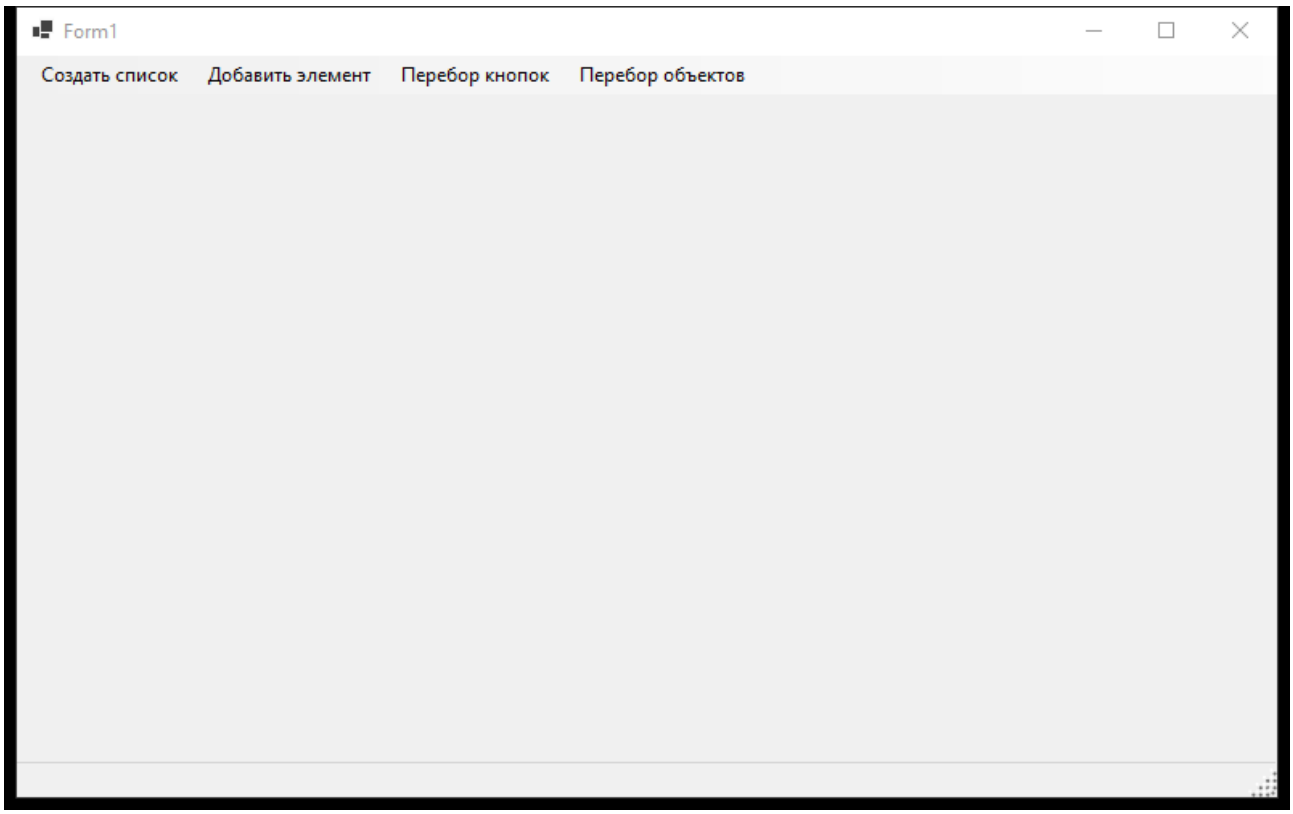


Рис. 1: Окно сразу после запуска

5.2. Добавление без списка

Шаги: не создавая список, выбрать «Добавить элемент».
Ожидание: панель пустая, в строке состояния «Список не создан.»
Факт: совпало, рисунок 2.

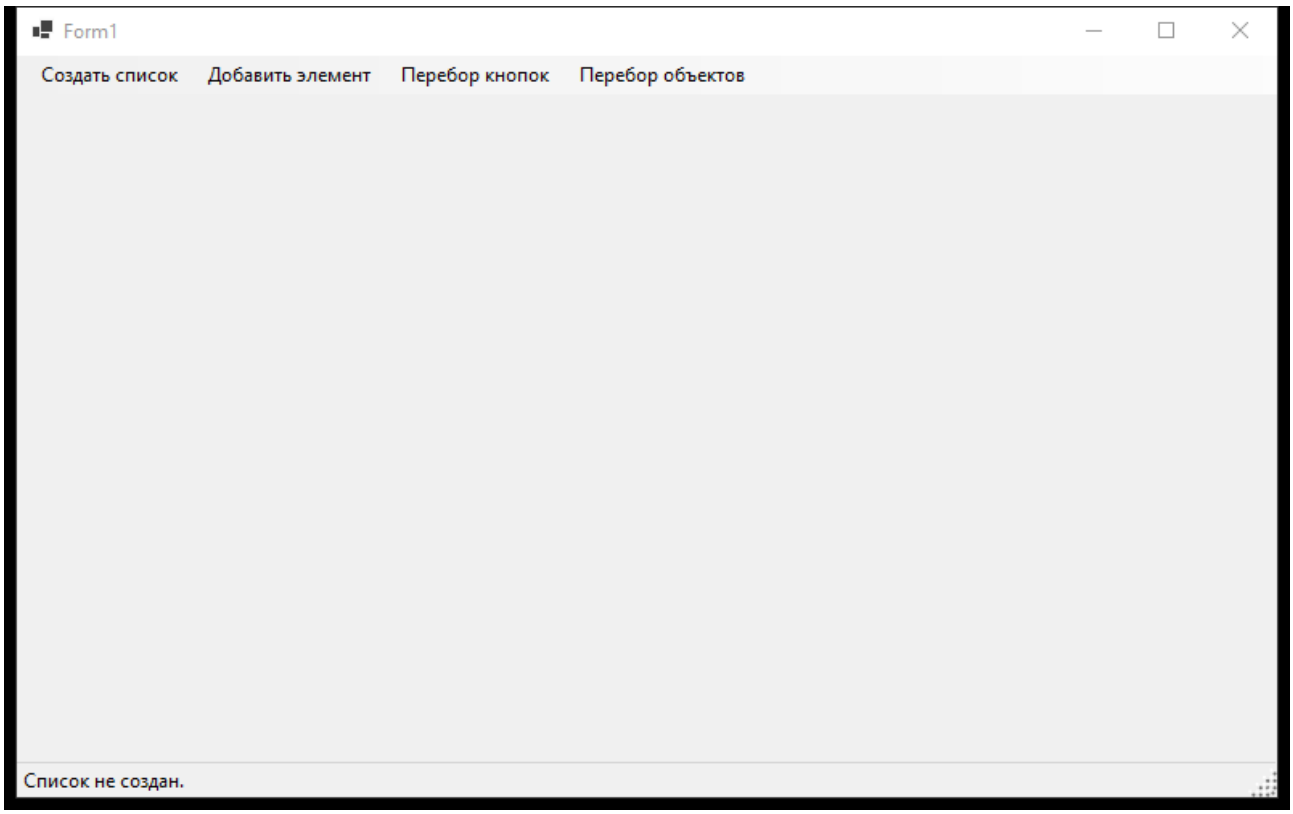


Рис. 2: Отказ добавить кнопку до создания списка

5.3. Создание списка

Шаги: выбрать «Создать список».

Ожидание: панель по-прежнему пустая, в строке состояния «Список создан.»

Факт: совпало, рисунок 3.

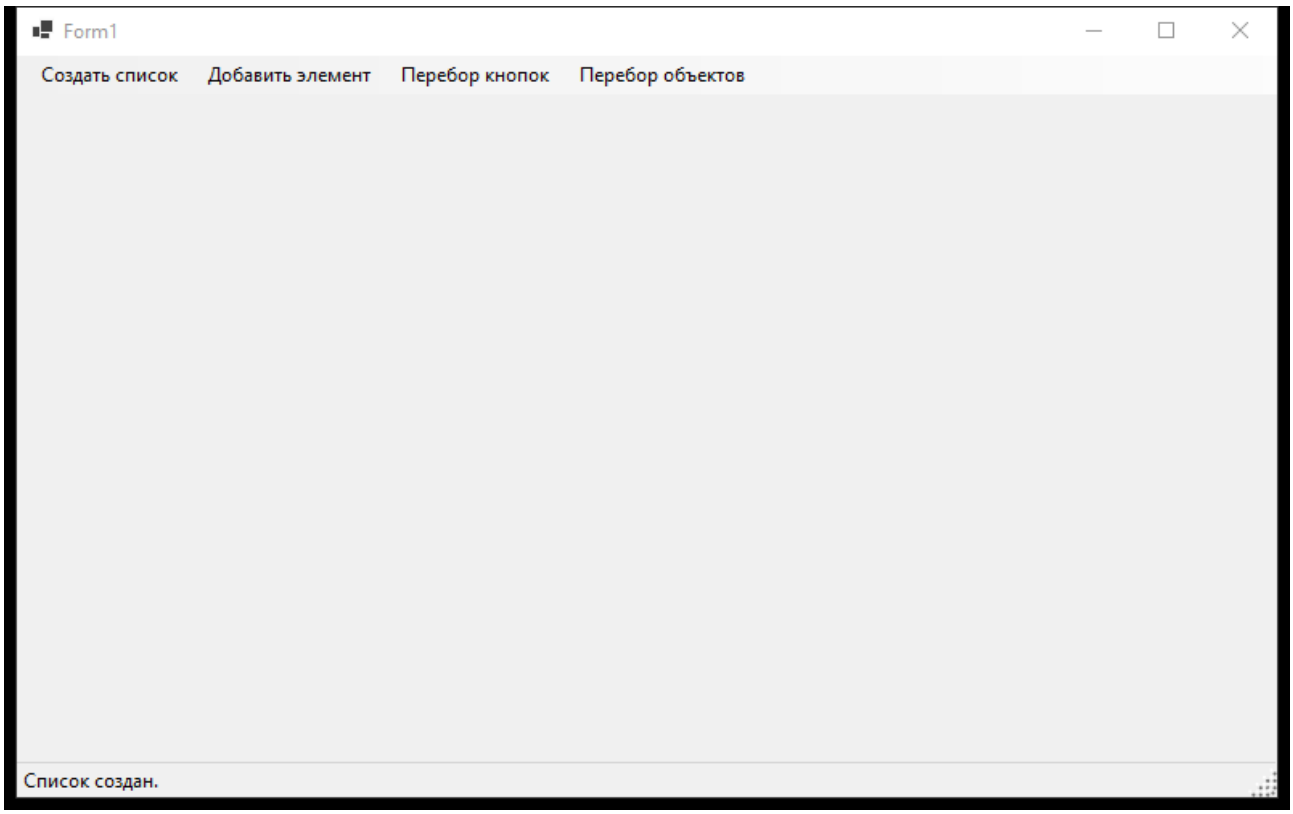


Рис. 3: Список создан, узлов ещё нет

5.4. Перебор пустого списка

Шаги: при только что созданном списке выбрать «Перебор кнопок».
Ожидание: «Список пуст.», исключение не возникает.
Факт: совпало, рисунок 4.

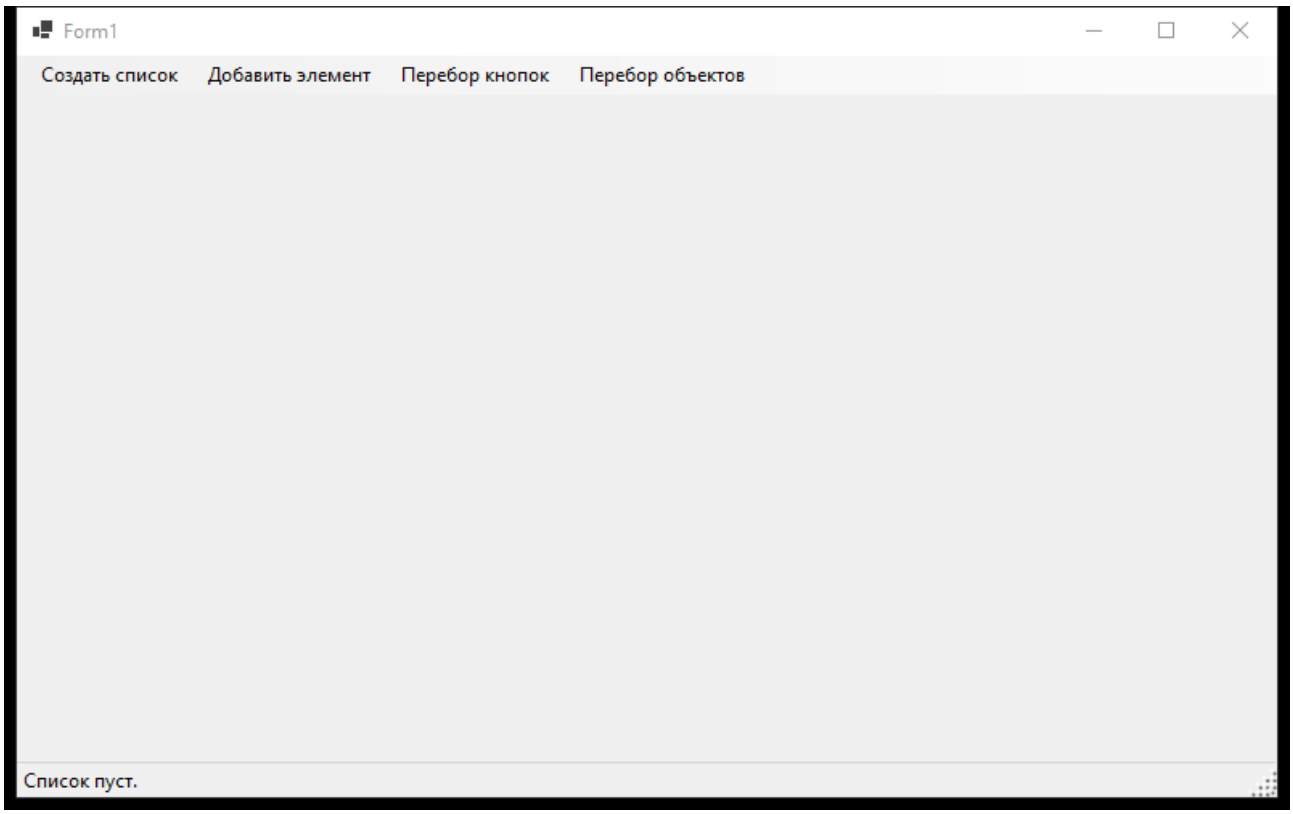


Рис. 4: Перебор не начат: в списке нет узлов

5.5. Ряд кнопок

Шаги: двадцать два раза выбрать «Добавить элемент». Отметить кнопки со смещением 80 и со смещениями от 160 до 400 включительно.

Ожидание: подписи равны смещению 0, 40, 80, …, 840. Кнопки шириной 30 не показывают все цифры чисел от 100. Внизу панели — горизонтальная прокрутка. Отмечены кнопка 80 и кнопки 160--400. Строка состояния пустая: добавление её очищает.

Факт: совпало, рисунок 5. На видимом участке подпись 120 обрезана до «12», 160 — до «16», 400 — до «40».

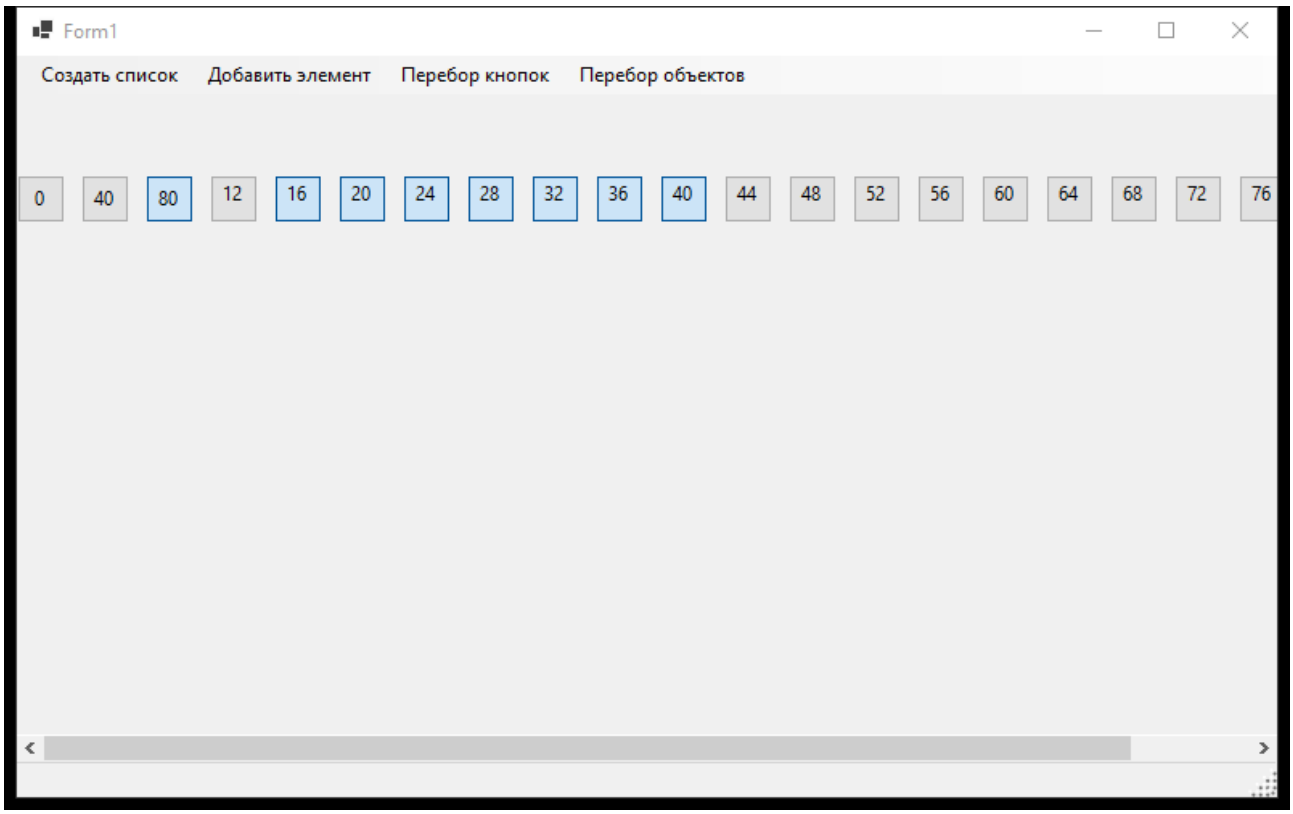


Рис. 5: Двадцать две кнопки, часть отмечена вручную

5.6. Перебор кнопок с головы списка

Шаги: выбрать «Перебор кнопок» и остановить проход на первом узле. Ожидание: голова списка — последняя добавленная кнопка, смещение 840. Панель прокручена вправо. Отмечена только она. Строка состояния — «Кнопка: 840».

Факт: совпало, рисунок 6. На кнопке из-за ширины 30 виден фрагмент «84».

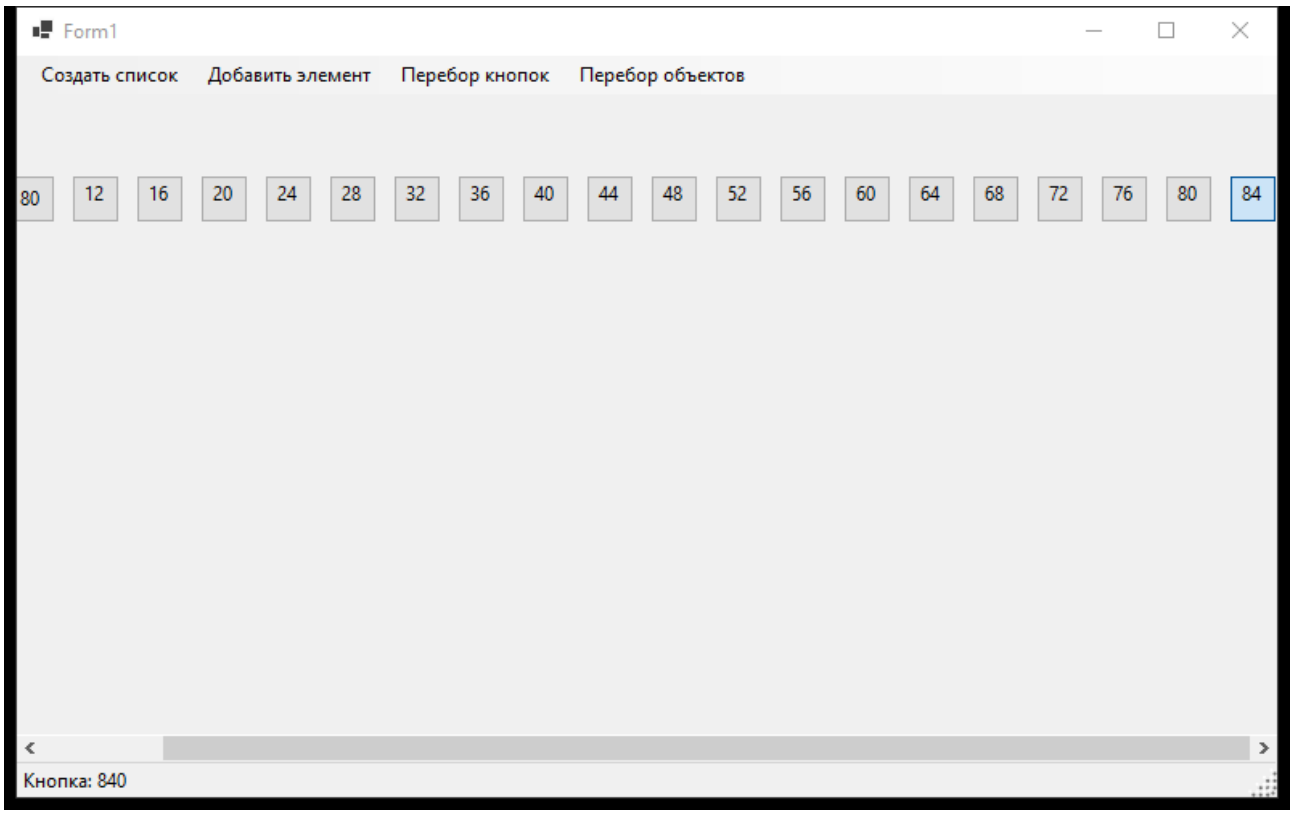


Рис. 6: Первый шаг необобщённого перечислителя

Шаги: сделать ещё шесть шагов того же прохода.

Ожидание: текущее смещение 840 - 6 * 40 = 600, строка «Кнопка: 600», отмечена одна кнопка.

Факт: совпало, рисунок 7.

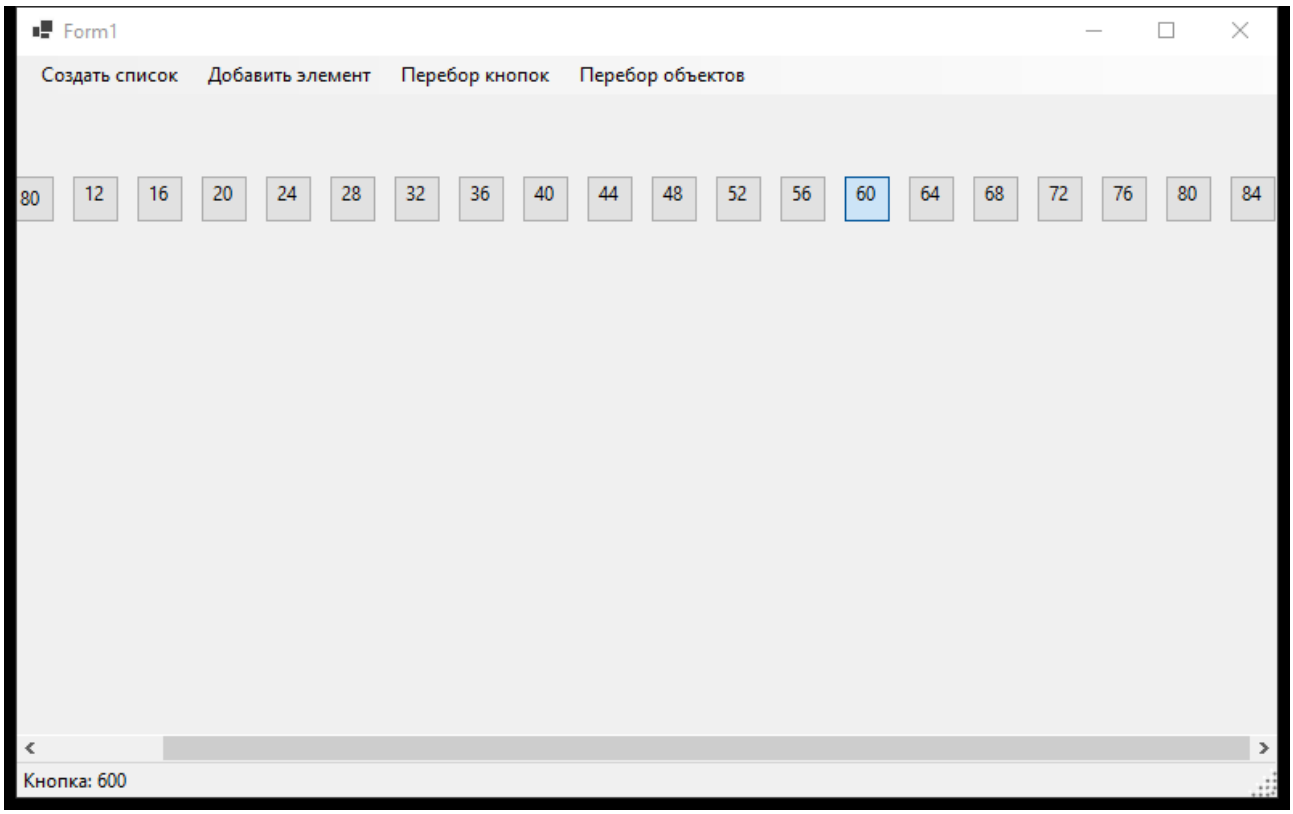


Рис. 7: Шестой шаг после головы: кнопка 600

5.7. Завершение прохода

Шаги: дойти до конца того же перебора.

Ожидание: строка состояния «O.K.», отметки вернулись к состоянию до команды (80 и 160--400), прокрутка снова показывает левый край.

Факт: совпало, рисунок 8.

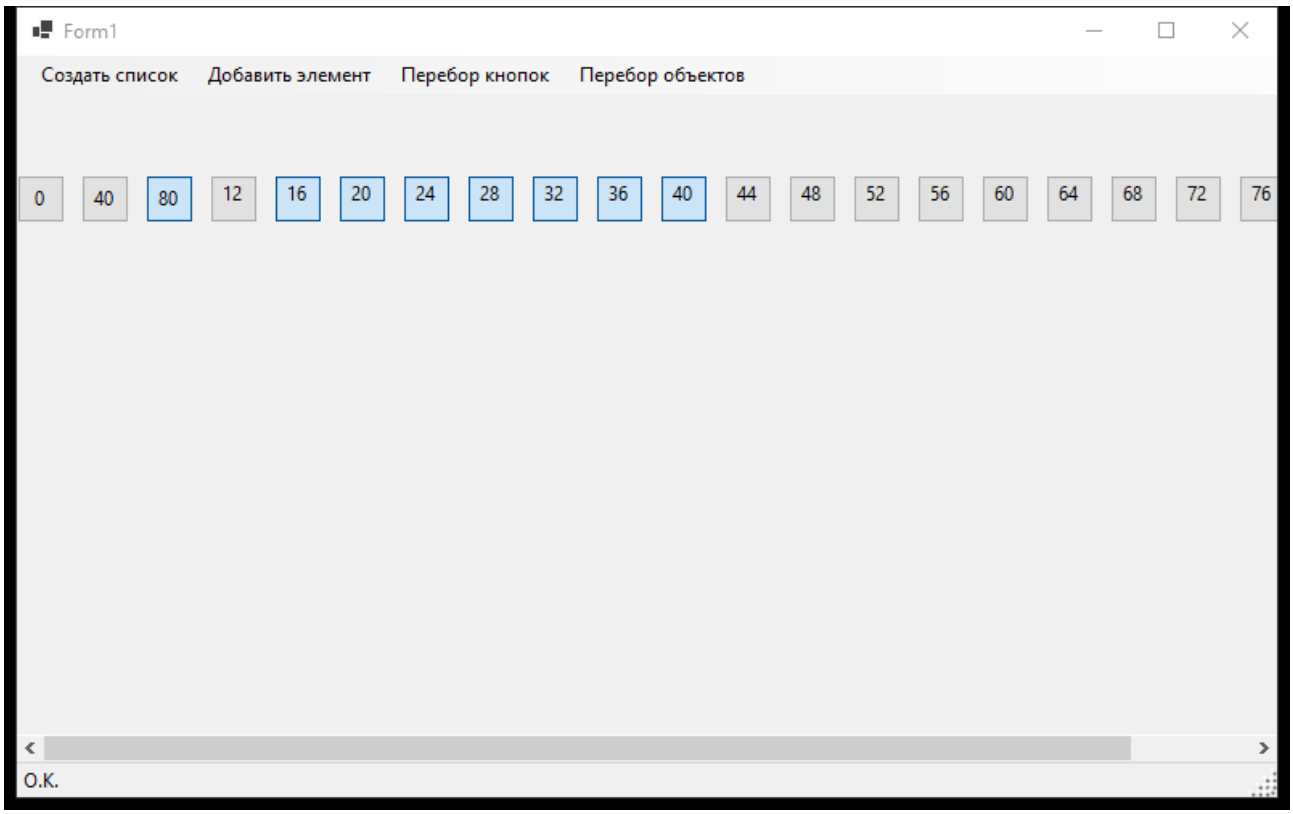


Рис. 8: После прохода отметки восстановлены

5.8. Перебор объектов

Шаги: выбрать «Перебор объектов» и перейти на третий узел.

Ожидание: порядок тот же, что у кнопок. Первый узел — 840, третий — 760.

Строка состояния — «Объект: 760», отмечена одна кнопка.

Факт: совпало, рисунок 9.

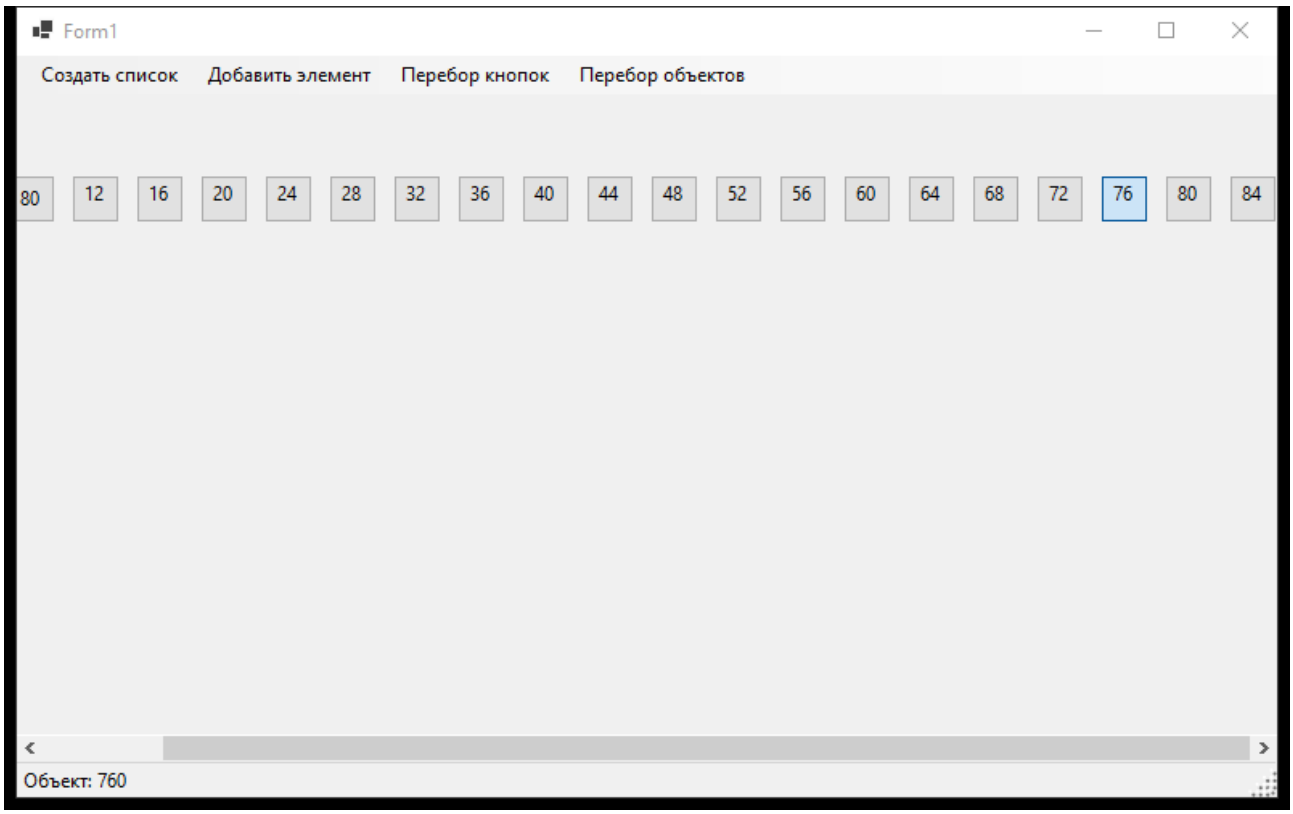


Рис. 9: Обобщённый перечислитель, третий узел

5.9. Новый список продолжает нумерацию

Шаги: завершить перебор, выбрать «Создать список», затем два раза «Добавить элемент» и прокрутить панель к новым кнопкам.

Ожидание: прежние двадцать две кнопки сняты. Счётчик после них равен 22, поэтому новые смещения — 880 и 920. Строка состояния пустая, потому что её очищает добавление.

Факт: совпало, рисунок 10. На кнопках видны обрезанные «88» и «92».

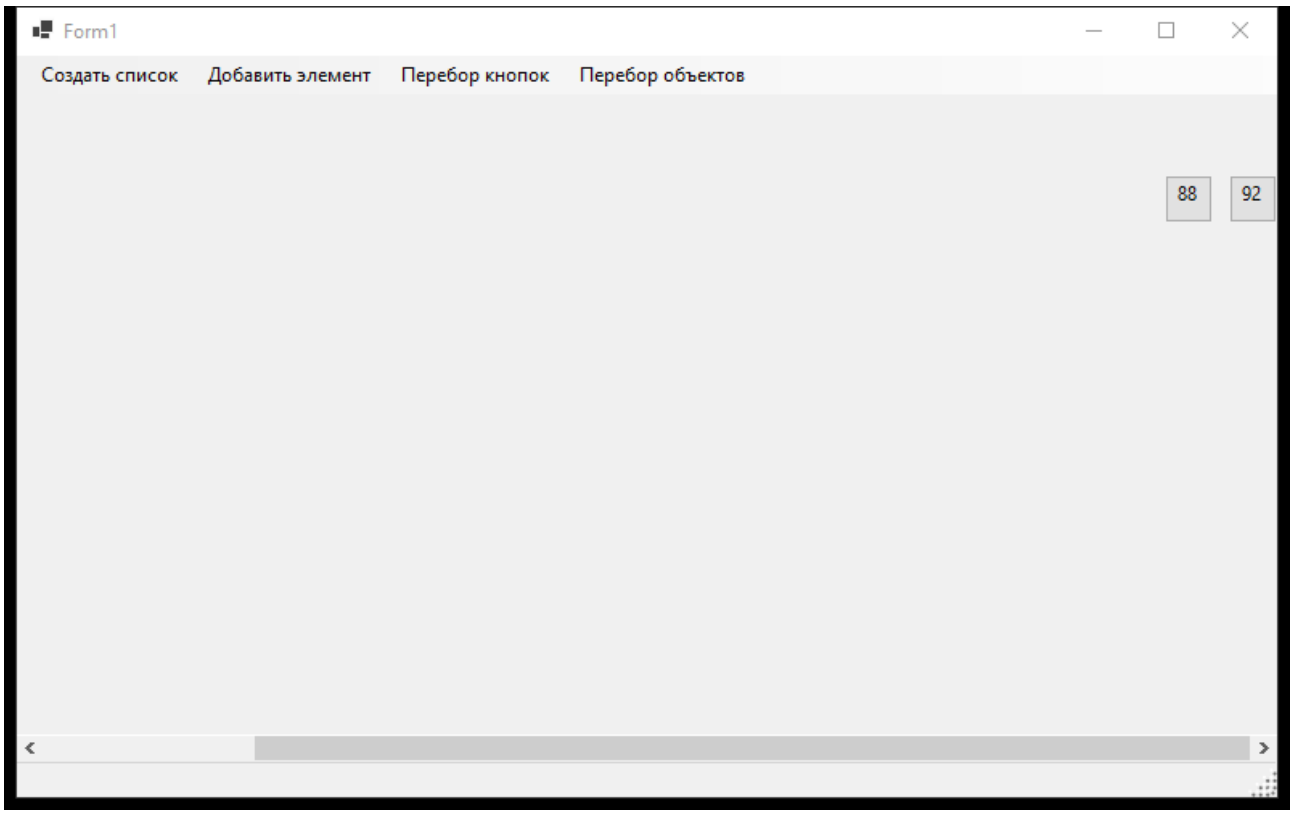


Рис. 10: Новый список: старые кнопки сняты, номера продолжаются

5.10. Сценарии, закрытые только проверкой перечислителя

Проверка	Ожидание	Результат
Current до MoveNext на пустом списке	исключение	пройден
MoveNext на пустом списке	false	пройден
Current после этого MoveNext	исключение	пройден
три вызова Append	номера $n, n+1, n+2$	пройден
голова и хвост после a, b, c	c, затем b, затем a	пройден
foreach по MiniList<int>	3, 2, 1	пройден
foreach по IEnumerable	та же последовательность	пройден
необобщённый Current	то же значение	пройден
Reset после полного прохода	повтор даёт прежний первый элемент	пройден
MoveNext за последним узлом дважды	оба раза false	пройден
два перечислителя одного списка	каждый хранит свой узел	пройден
Dispose	исключения нет	пройден

6. Документация

6.1. Руководство оператора

Запуск: dotnet run --project Lab3. Открывается одно окно. Сначала выбирают «Создать список». Пока этой команды не было, «Добавить элемент» и обе команды перебора только пишут причину отказа в строку состояния. «Добавить элемент» ставит справа следующую кнопку. Если ряд уходит за край панели, внизу появляется горизонтальная прокрутка. Щелчок по кнопке отмечает её и снимает отметку: это обычное поведение CheckBox с видом кнопки и к списку не относится. «Перебор кнопок» и «Перебор объектов» по очереди отмечают узлы списка, начиная с последней добавленной кнопки и двигаясь к первой.

В строке состояния видно полное смещение, даже если на узкой кнопке число обрезано. Когда проход закончен, прежние отметки возвращаются и появляется «O.K.» Повторная команда начинает проход сначала. «Создать список» во время прохода прерывает его и убирает кнопки.

6.2. Сборка

Нужен SDK .NET 10 с поддержкой Windows Forms. Из каталога лабораторной:

```
dotnet run —project Lab3.Tests
dotnet run —project Lab3
```

Отчёт собирается XeLaTeX из каталога report (нужны polyglossia, fontspec, listings):

```
xelatex lab3_report.tex
xelatex lab3_report.tex
```

Второй проход нужен для оглавления.

6.3. Ограничения

Удалить один узел или сохранить список на диск нельзя: в задании этого нет. Подпись на кнопке обрезается, потому что ширина 30 взята из образца. Перебор идёт от последней кнопки к первой, потому что Append вставляет узел в голову. Новый список не начинает нумерацию с нуля: статический счётчик образца этого не делает, и сброс в программу не добавлялся. Во время прохода отметка кнопки занята подсветкой текущего узла; после «O.K.» возвращается то, что оператор отметил сам.

6.4. Состав поставки

Путь	Назначение
Lab3.sln	решение
Lab3	окно и MiniList<T>
Lab3.Tests	19 проверок перечислителя
Протокол_тестирования.txt	план, результаты и заключение прогона
report/lab3_report.tex	настоящий отчёт
report/images	снимки сценариев
report/_capture	повторная съёмка окна

Заключение

По образцу лабораторной собрано окно с меню, прокручиваемой панелью кнопок и строкой состояния. Коллекция CheckBox хранится в односвязном списке MiniList<T> с вставкой в голову. Список отдаёт элементы через обобщённый и необобщённый перечислители. «Перебор кнопок» и «Перебор объектов» показывают один и тот же порядок узлов и различаются интерфейсом, через который этот порядок получен. По окончании прохода строка состояния содержит «O.K.», а отметки кнопок возвращаются. Контракт MoveNext, Current, Reset и Dispose закрыт 19 проверками. Поведение окна закрыто десятью снимками: отказ до создания списка, пустой список, ряд кнопок, оба вида перебора и новый список с продолжением нумерации.